

A modern Rust blockchain for games and metaverse economies should treat **assets as first-class objects**, keep **gameplay off the hot path**, and settle **ownership + value** on a high-throughput L1. The design below is a synthesis of what works in production (object-centric models like Sui, parallel execution like Solana/Aptos, hybrid on/off-chain game loops) rather than a clone of any one chain.

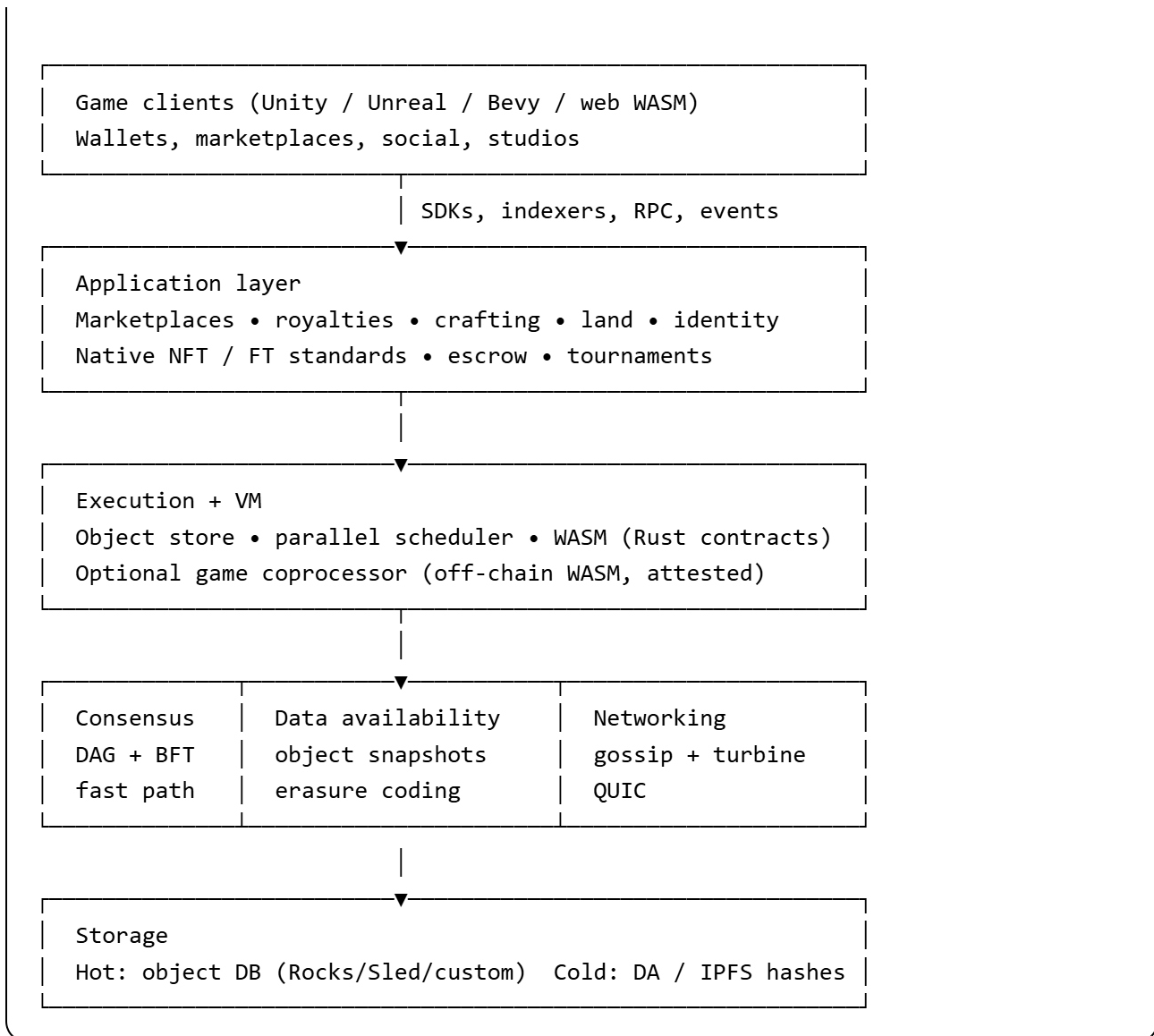
Design goals

- Sub-second finality for simple asset moves (equip, trade, craft).
- Tens to hundreds of thousands of TPS when transactions do not contend on the same objects.
- Near-zero fees for in-game micro-actions.
- True ownership of items, land, characters, and currencies that can move across titles.
- Gameplay that still feels like a game (60+ FPS, physics, AI) — not a wallet confirmation every click.
- One language for protocol and contracts: **Rust**.

Gameplay stays off-chain or in a WASM coprocessor; the chain is the **ledger of record** for the digital economy.

Layered architecture





1. Object-centric state (the important part for games)

Do **not** store everything as “account + balance.” Treat every asset as an **object** with:

- globally unique ID
- owner (address, another object, or shared)
- type + version
- capabilities (transfer, mutate, wrap)

Examples: a sword object owned by a character object owned by a player. Independent objects execute in parallel with no global lock. Shared objects (a marketplace pool, a raid boss) go through consensus. This matches how games already think about inventories.

2. Consensus and execution (Rust node)

- **Fast path:** single-owner object txs skip full ordering; certify and commit in ~100–400 ms.
- **Slow path:** shared objects use a DAG mempool (Narwhal-style) + BFT (Bullshark / Mysticeti-class) so data dissemination is not the bottleneck.
- **Parallel execution:** scheduler uses declared object IDs (explicit, like Solana accounts) plus ownership so non-overlapping txs run concurrently. Optimistic STM (Block-STM style) as fallback for shared state.
- **Finality:** deterministic, no long reorgs — required if items have real value.

The validator, networking, and execution engine are a Rust workspace (Tokio, QUIC, blst/ed25519, custom object DB). No EVM unless you add it as an optional sidecar.

3. Smart contracts in Rust

Two complementary runtimes:

Runtime	Use	Why
On-chain WASM	Economy: mint, trade, royalties, staking, land deeds	Sandboxed, portable, Rust → wasm32
Native / BPF-like programs	Ultra-hot paths (order books, tick engines)	Solana-style performance
Off-chain WASM coprocessor	Combat resolution, physics, matchmaking, RNG with commit-reveal	Near-native speed; only hashes / outcomes land on-chain

Contracts should express **object types and capabilities**, not raw storage slots. That keeps asset composition (bag of items, nested NFTs) cheap and safe.

4. Hybrid game loop (non-negotiable)

Lives on-chain	Lives off-chain / coprocessor
Ownership, balances, marketplace fills	Input, rendering, physics
Crafting recipes that mint/burn assets	Frame-by-frame combat
Land deeds, rent, royalties	Chat, presence, pathfinding
Tournament prizes, escrow	Anti-cheat sensors (attest results)

Backend / sequencer cluster runs the same Rust WASM game handlers, posts periodic checkpoints or fraud/validity proofs. Players never wait on a block for a jump.

5. Digital economy primitives (protocol-level)

Build these as native object types, not ad-hoc contracts:

- **FT + NFT + dynamic NFT** (stats that mutate without reminting).
- **Composability**: objects own objects (character owns loadout owns gems).
- **Royalties / creator cuts** enforced at transfer.
- **Escrow + atomic swap** for P2P and studio marketplaces.
- **Fee market**: priority fees + storage rent so abandoned items do not bloat state.
- **In-game currency** as a first-class coin object, optionally bridged.
- **Identity**: passkeys / zkLogin so a new player is not forced through seed phrases on day one.

Media (meshes, textures) stays off-chain; the object stores a content hash + URI.

6. Data, indexers, and studio APIs

- Object DB + Merkle / verkle snapshots for light clients and rollbacks.
- Dedicated **game indexer** (Rust): inventory views, activity feeds, leaderboards, “what changed since last frame.”
- Event bus (WebSocket / gRPC) so engines subscribe to *their* objects only.
- Oracles only where needed (fiat ramps, external randomness with VRF).

7. Modularity and interoperability

Keep consensus, execution, and DA separable so you can later:

- run an app-specific rollup for one title,
- use an external DA layer for cheap media commitments,
- bridge assets to Ethereum / Solana without rewriting the object model.

Bridges should move **objects**, not just wrapped ERC-721s.

8. Security model for valuable items

Games with real money attract duplication and admin keys. Protocol-level rules:

Games with real money attract duplication and admin keys. + recover revert rules.

- No silent mint: supply changes are typed object operations.
- Shared objects have explicit shared-vs-owned modes.
- Admin / “studio” keys are time-locked or DAO-gated for economy parameters.
- Deterministic WASM + attested coprocessor outputs; never trust a single game server for mint.
- Rate limits and storage rent against spam inventories.

Suggested Rust crate map (implementation sketch)

- `node` — validator binary
- `consensus` — DAG + BFT
- `execution` — object store + scheduler
- `vm-wasm` — contract runtime
- `objects` — NFT/FT/land types
- `p2p` — QUIC gossip
- `rpc / indexer`
- `sdk` — Rust + FFI for engines
- `coprocessor` — optional verifiable game runtime

Contracts: `#[object]` macros, capability types, tests against an in-memory object store (same pattern as Anchor / Sui Move tests, but Rust).

What not to do

- Put every combat tick on L1.
- Use a global account model as the only state shape.
- Make players pay Ethereum-mainnet fees for a potion.
- Hide the studio as an unbounded mint authority.
- Invent a new contract language when Rust + WASM already exists.

This stack is “modern” because it matches how games actually work: **many independent objects moving at once**, a thin settlement layer for the economy, and Rust from node to contract to (optionally) game logic. Sui-style objects plus Solana-class parallelism plus a WASM coprocessor is the combination that currently fits metaverse digital economies best